

Requirement Engineering Tasks

1. Inception – *Basic Understanding*

Initiating the Requirement Engineering Process

➔ How Requirements Originate:

- Identifying a **business need**.
- Discovering a **new market** or **service opportunity**.
- Developing a **working project description**.

Steps to Initiate Requirement Engineering

✓ a) Identifying the Stakeholders

- **Stakeholder** → Anyone who benefits from the system (directly or indirectly).
- Possible Stakeholders:
 - Business operations managers
 - Product managers
 - Marketing team
 - Internal/external customers
 - End-users
 - Consultants
 - Product engineers
 - Software engineers
 - Support and maintenance team

➔ Role of Requirement Engineer:

- Prepare a list of contributors who provide input during the requirements phase.

✓ b) Recognizing Multiple Viewpoints

- Each stakeholder explores the system requirements from their **own perspective**.
- May lead to **inconsistent** or **conflicting** requirements.
- Requirement engineer must **categorize** the information collected.

✓ c) Working Towards Collaboration

- Stakeholders with different views must work together.
- Engineer helps to **resolve conflicts** and **align goals**.

✓ d) Asking the First Questions

- Establish communication with customers.
- Sample questions:
 - Who is behind the request?
 - What is the business benefit?
 - Are there alternative solutions?
 - How will success be measured?
 - What constraints must be considered?

2. Elicitation – *Clear Understanding*

Goal:

- Collect clear, complete, and structured requirements from stakeholders.

➡ Common Techniques:

- Collaborative Requirement Gathering
- Quality Function Deployment (QFD)
- User Scenarios (Use Cases)
- Prototypes

3. Elaboration – *Refinement*

- Further refining the gathered requirements.
- Developing a clear understanding of:
 - System functions
 - Performance expectations
 - System behavior
 - Constraints

Outcome:

- Well-defined system requirements.

4. Negotiation – *Settlement of Conflicts*

- Resolve conflicts among stakeholders.
- Balance between:
 - Functionality
 - Performance
 - Cost
 - Timeline
- Goal → "Win-Win" outcome for all stakeholders.

5. Specification – *SRS (Software Requirement Specification)*

- Formal document that includes:
 - Functional requirements
 - Non-functional requirements
 - Constraints
 - Performance details
 - Design limitations

Purpose:

- Clear reference for developers, testers, and stakeholders.

6. Validation – *Formal Technical Review (FTR)*

- Ensures requirements are:
 - Complete
 - Consistent
 - Testable
 - Feasible
- **FTR team** validates the requirements through reviews.

7. Management – *Change Management*

- Requirements may change due to:
 - Business changes
 - Technology shifts
 - Market conditions
- Requirement Engineer must track and manage changes.

Summary:

- ✓ **Inception** → Stakeholder Identification & Initial Understanding
- ✓ **Elicitation** → Collect Requirements
- ✓ **Elaboration** → Refine Requirements
- ✓ **Negotiation** → Resolve Conflicts
- ✓ **Specification** → Create SRS Document
- ✓ **Validation** → Review by Technical Team
- ✓ **Management** → Handle Requirement Changes

Working Towards Collaboration

- Customers and stakeholders should **collaborate** with software engineers to ensure project success.
- Requirements must be:
 - **Sorted out** to align with common goals.
 - **Resolved** if there are any inconsistencies or conflicts.

Asking the First Questions

➔ Questions at the start should be **context-free** (unbiased and open-ended).

Objective:

- Establish communication between stakeholders and developers.
- Identify **all stakeholders**.
- Understand the problem better.
- Break the ice and initiate communication.

☑ First Set of Questions – *Identify Stakeholders & Goals*

- Who is behind the request for this work?
- Who will use the solution?
- What will be the economic benefit of a successful solution?
- Is there any other source for the solution you need?

➔ Purpose:

- Identify key stakeholders.
- Understand the business value and need for the solution.

Next Set of Questions – *Understand the Problem*

- How would you categorize "good output" from a successful solution?
- What problems will this solution address?
- Can you show me the business environment where the solution will be used?
- Are there any special performance issues or constraints?

Purpose:

- Clarify the problem space.
- Identify constraints and performance expectations.

Final Set of Questions – *Evaluate Communication & Completeness*

- Are you the right person to answer these questions?
- Are your answers official?
- Are my questions relevant to the problem?
- Am I asking too many questions?
- Can anyone else provide additional information?
- Should I be asking you anything else?

Purpose:

- Ensure accurate and complete information.
- Improve communication and understanding.

Summary:

- ✓ **Collaboration** – Align stakeholders and engineers toward common goals.
- ✓ **First Questions** – Identify stakeholders and business goals.
- ✓ **Next Questions** – Understand the problem and environment.
- ✓ **Final Questions** – Ensure communication clarity and completeness.

Eliciting Requirements

➔ The process of **collecting and defining** requirements from stakeholders and customers.

Elicitation Format

1. Collaborative Requirement Gathering
2. Quality Function Deployment (QFD)
3. User Scenarios (Use Cases)
4. Elicitation Work Products

1. Collaborative Requirement Gathering

Goal: Identify problems and issues to define accurate requirements.

➔ Guidelines:

- A **team of stakeholders** and developers work together.
- Meetings should be conducted with:
 - **Formal agenda** – Topics, date, time, and venue should be shared in advance.
 - **Rules for preparation and participation** – Everyone should know the goals and expectations.
 - **Facilitator** – A person to manage the meeting and keep discussions on track.
 - **Definition Mechanism** – To define and document the gathered information.

☑ 2. Quality Function Deployment (QFD)

- **QFD** focuses on understanding customer requirements and converting them into technical requirements.
- Goal:
 - Maximize customer satisfaction.
 - Translate customer demands into design specifications.

☑ 3. User Scenarios (Use Cases)

- Define how the user will interact with the system.
 - Helps developers to understand:
 - **Expected system behavior**
 - **Edge cases**
 - **Possible failures**
- ➡ Example:
- User logs in → Views dashboard → Makes payment → Logs out.

☑ 4. Elicitation Work Products

➡ Outcome of Elicitation Process:

- **Statement of need** and feasibility.
- **Bounded statement of scope** – Clearly define the system boundaries.
- **List of stakeholders** – Who is involved and how they are affected.
- **Description of technical environment** – Operating system, hardware, etc.
- **List of functional and non-functional requirements** – Sorted function-wise.
- **Set of usage scenarios** – Use cases or workflow examples.
- **Prototypes** – If developed, they define initial requirements.

Negotiating Requirements

- Customer must balance:
 - **Functionality**
 - **Performance**
 - **System constraints**
 - **Cost and time**

Goal:

- Develop a **project plan** that aligns customer expectations with technical feasibility.
- **Win-Win** – Best negotiations satisfy both customer and developer needs.

Validating Requirements

- As each element of the analysis model is created, it should be checked for:
 - **Consistency** – Does it align with the overall objective?
 - **Omissions** – Is any key information missing?
 - **Ambiguity** – Is it clearly defined?

FTR (Formal Technical Review) checks for:

- ✓ Consistency of each requirement with overall objectives.
- ✓ Proper specification of requirements.
- ✓ Necessity of each requirement.
- ✓ Clear source of each requirement.
- ✓ No conflicting requirements (information, function, behavior).
- ✓ Technical feasibility of requirements.
- ✓ Testability of requirements.

Building the Analysis Model

→ Text alone is not enough to communicate requirements effectively.

→ Use a **combination of text and diagrams** to represent:

- **Data requirements**
- **Functional requirements**
- **Behavioral requirements**

Goal:

- Increase clarity and completeness.
- Improve consistency.
- Surface errors and missing elements early.

→ **Multiple Dimensions:**

- View the model from different perspectives → Higher chance of spotting issues.
- Ensures the solution covers all business and technical needs.

☑ **Summary:**

- ✓ **Elicitation** – Gathering complete and structured requirements.
- ✓ **Negotiation** – Aligning customer goals with technical limits.
- ✓ **Validation** – Ensuring consistency and completeness.
- ✓ **Analysis Model** – Text + diagrams to improve clarity and correctness.

Building the Analysis Model

→ Written communication alone is not sufficient for defining software requirements.

→ A combination of **text and diagrams** provides a clearer and more structured view of:

- **Data requirements**
- **Functional requirements**
- **Behavioral requirements**

☑ Role of the Software Engineer

- Builds the model using elicited requirements.
- Helps validate software requirements from multiple viewpoints.
 - ➔ Increases the chance of finding:
 - Errors
 - Inconsistencies
 - Omissions

Analysis Rules of Thumb

- 1. Focus on Visible Requirements**
 - a. Focus on the business and problem domain requirements.
- 2. High Level of Abstraction**
 - a. Don't get into low-level details during analysis.
- 3. Add Value**
 - a. Every element should increase understanding and provide insight into:
 - i. Information domain
 - ii. Function
 - iii. Behavior
- 4. Ignore Infrastructure Details**
 - a. Delay infrastructure and non-functional details until the design phase.
- 5. Minimize Coupling**
 - a. Reduce dependency between components.
- 6. Keep It Simple**
 - a. Avoid unnecessary complexity.

Analysis Modeling Approaches

☑ 1. Structured Analysis

- **Data and processes** are treated as separate entities.
- Models include:

- **Data objects** – Attributes and relationships are defined.
- **Processes** – How data flows and transforms within the system.
 - ➔ Focused on **flow-oriented modeling**.

☑ 2. Object-Oriented Analysis (OOA)

- Focuses on:
 - **Definition of classes**
 - **Collaboration between classes** to meet customer requirements.
- Uses:
 - **UML (Unified Modeling Language)**
 - **Unified Process**
 - ➔ More adaptable to complex systems with reusable components.

Analysis Model Components

- ➔ Only include modeling elements that **add value** to the analysis process.
- ➔ The specific content of each element may vary based on the project type and scope.

☑ Summary:

- ✓ Combine text and diagrams for better communication.
- ✓ Follow analysis rules to maintain clarity and consistency.
- ✓ Use structured or object-oriented analysis based on project needs.
- ✓ Keep the model focused and simple.